==========

==========

CHAPTER 2 — ADDRESSING & VIRTUAL ADDRESS STRUCTURE

==========

==========

A complete, maximum-detail breakdown of how ARMv8 defines, interprets, and manages virtual addresses.

****

****

2.1 — Introduction to Addressing in ARMv8

****

****

ARMv8 uses a **64-bit virtual address space**, but not all 64 bits are used. The MMU interprets virtual addresses according to:

- **TCR_EL1** (Stage-1)
- **TCR_EL2** (Stage-2)
- **granule size**

- **translation regime** (EL0/EL1/EL2)

Virtual addresses are divided into:

- **TTBR0 region** (typically user space)
- **TTBR1 region** (typically kernel space)

The split is controlled by **TCR_EL1.T0SZ** and **TCR_EL1.T1SZ**.

****

****

2.2 — Virtual Address Bit Ranges

****

****

ARMv8 supports multiple VA sizes:

- 48-bit VA (most common)
- 52-bit VA (extended)
- 56-bit VA (rare, server-class)

Example: 48-bit VA

```
[ VA[47:0] ]  usable
[ VA[63:48] ] must be sign-extended
```

Teaching Note

Sign-extension ensures that virtual addresses remain canonical. Non-canonical addresses generate faults.

****

****

2.3 — Canonical Addressing

****

****

ARM requires that unused upper bits of VA be:

- **all zeros**, or
- **all ones**

depending on the configured VA size.

If VA is 48 bits:

```
Valid VA:
  0x0000_0000_0000_0000  → 0x0000_FFFF_FFFF_FFFF
Invalid VA:
  0x0001_0000_0000_0000  (upper bits not sign-extended)
```

Implementation Note

Canonicality is checked *before* translation. Invalid canonicality → **Address Size Fault**.

****

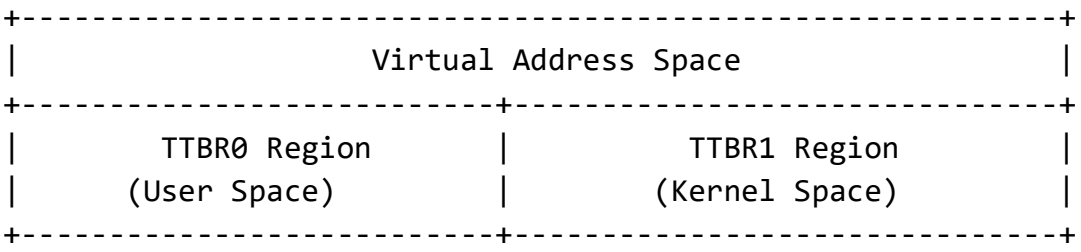
****

2.4 — TTBR0 vs TTBR1 Virtual Address Regions

****

****

The virtual address space is split into two regions:



Controlled by:

- **TCR_EL1.T0SZ** → size of TTBR0 region
- **TCR_EL1.T1SZ** → size of TTBR1 region

Example Split (Typical Linux/QNX)

```
TTBR0: 0x0000_0000_0000_0000 → 0x0000_FFFF_FFFF_FFFF
TTBR1: 0xFFFF_0000_0000_0000 → 0xFFFF_FFFF_FFFF_FFFF
```

Teaching Note

TTBR1 is global and shared across all processes. TTBR0 changes per process (ASID).

****

****

2.5 — TCR_EL1 Controls VA Size

****

****

TCR_EL1 contains:

- **T0SZ** → size of TTBR0 VA region
- **T1SZ** → size of TTBR1 VA region

Formula:

```
VA_bits = 64 - TnSZ
```

Example

If T0SZ = 16:

```
VA_bits = 64 - 16 = 48 bits
```

****

****

2.6 — Granule Size and VA Interpretation

****

****

Granule size determines:

- page size
- page table level count
- VA bit slicing

Supported granules:

- **4 KB** (most common)
- **16 KB**
- **64 KB**

VA slicing for 4 KB granule:

```
VA[47:39] → L0 (optional)
VA[38:30] → L1
VA[29:21] → L2
VA[20:12] → L3
VA[11:0]  → page offset
```

Teaching Note

Granule size affects *everything*: page table depth, descriptor interpretation, memory alignment, and TLB behavior.

****

****

2.7 — Address Regions and Translation Regimes

****

****

ARMv8 defines multiple translation regimes:

Regime	Used By	Translation
EL0/EL1	User + Kernel	Stage-1
EL2	Hypervisor	Stage-2
EL3	Secure Monitor	Secure translation

Implementation Note

EL2 can disable Stage-1 for certain accesses (rare, used in hypervisor-only systems).

****

****

2.8 — Virtual Address Generation

****

****

The CPU generates VA for:

- instruction fetch
- load/store
- atomic operations
- system register access (mapped)
- stack operations
- PC-relative addressing

Teaching Note

The MMU sees *every* memory access except:

- system registers
- debug registers
- certain EL3 secure accesses

****

****

2.9 — Virtual Address Alignment

****

****

Alignment rules depend on:

- access size
- memory type
- architecture version

Examples:

- 8-byte load → VA must be 8-byte aligned
- Device memory → stricter alignment rules

Misalignment → **Alignment Fault**.

****

****

2.10 — Virtual Address Fault Types

****

****

Faults generated before translation:

Address Size Fault

VA not canonical.

Translation Fault

No valid descriptor found.

Access Flag Fault

AF bit not set.

Permission Fault

AP or S2AP denies access.

Alignment Fault

VA not properly aligned.

****

****

2.11 — Virtual Address to TTBR Mapping

****

****

The MMU determines which TTBR to use:

```
if VA in TTBR0 region:
    use TTBR0_EL1
else:
    use TTBR1_EL1
```

Implementation Note

TTBR1 mappings are typically global kernel mappings.

****

****

2.12 — Virtual Address Example (Full Walkthrough)

****

****

Given:

- VA = 0x0000_7FFF_1234_5678
- T0SZ = 16 → 48-bit VA
- Granule = 4 KB

Step 1 — Canonicality

Upper bits = 0 → valid.

Step 2 — TTBR selection

VA < split → TTBR0_EL1.

Step 3 — VA slicing

```
L1 index = VA[38:30]
L2 index = VA[29:21]
L3 index = VA[20:12]
Offset   = VA[11:0]
```

Step 4 — Stage-1 walk

Descriptor → IPA.

Step 5 — Stage-2 walk

Descriptor → PA.

****

****

2.13 — Chapter Summary

****

****

This chapter covered:

- VA structure
- canonical addressing
- TTBR0/TTBR1 split
- granule size effects
- VA slicing
- alignment rules
- fault types
- full VA → TTBR mapping example